

Dhaka Tesla Pool · Architecture

Browser → Next.js (Vercel) → NestJS API (Render) → PostgreSQL (Neon). En-route pooling on fixed routes. One monolith API, one database, no queues or caches: consistency comes from row locks and constraints.

REQUEST FLOW · HAPPY PATH

- 1

Nusrat requests Banani → Mohakhali

POST /api/rides from the passenger screen
- 2

Next.js forwards /api/* to the API

same origin, so the cookie stays first-party
- 3

Guards check session and role

then ownership inside the service
- 4

RidesService checks and saves

route fit, then REQUESTED with the Tk 75 estimate
- 5

Auto-join tries the nearest Tesla on the way

none on her route yet, so the ride waits
- 6

Jashim takes the suggested route

only routes through his car · then accept
- 7

Vehicle lock → re-check → seat

R1–R4, compare-and-set, seats_taken + 1
- 8

Rafiq and Shirin join

Rafiq at Banani, Shirin on the way at Mohakhali
- 9

Stop by stop: arrive, pick up, drop off

fares Tk 60, 72, 108 · Jashim earns Tk 180
- 10

Every ride status change → ride_events

and a hint to open screens over SSE

- LEGEND
- request (synchronous HTTP / SQL)
- - - - ->

 live updates (SSE), polling fallback
-

 build and deploy
- core of pooling / integrity
- 1

 step in the happy path

1 BROWSER

passenger and driver use the same web app

Login · Sign up

· passenger or driver, chosen first

· driver: NID/passport, licence, car

1 Passenger screen /ride

· request · route with the car · fare

· co-riders · cancel (Tk 20 if late) · history

6 Driver screen /driver

· suggested route · online · requests

· stop actions · earnings · past trips

2 WEB

Next.js 16 on Vercel

App Router pages

· client components ('use client')

· role-based redirects

TanStack Query

· refetch on each SSE hint (poll if down)

· loading / error / empty states

2 /api/* rewrite proxy

· forwards to the API URL

· first-party cookie, no CORS

3 API

NestJS 12 on Render · controller → service → repository · pure, unit-tested rules in fares/ (fare, earnings, cancellation) and pooling/ (route-plan, matching, route-suggestion)

Cross-cutting: helmet · cookie-parser · ValidationPipe (whitelist) · throttler 5/min on signup + login · pino JSON logs + request id · env validation at boot · RideErrorFilter { code, message } · SSE hints after each action

3 Guards: SessionAuthGuard (cookie → token hash → session) → RolesGuard @Roles(PASSENGER | DRIVER) → ownership check in the service (403 NOT_YOUR_RIDE)

AUTH

Controller

· POST /auth/signup/{passenger,driver}

· POST /auth/login · logout · GET /me

Service

· AuthService: bcrypt, session token

· 12 h httpOnly cookie

UsersRepository

· + SessionsRepository

GEOGRAPHY

Controller

· GET /zones · GET /routes

· GET /zones/id/destinations

Service

· GeographyService: zones, routes,

· destinations (route fit)

GeographyRepository

· + distance lookup for rides

RIDES (PASSENGER)

Controller

· POST /rides · /rides/id/cancel

· GET /rides/current · /rides

· GET /rides/id

Service

· RidesService: validate, route fit, estimate,

· one ride · cancel rules · late fee

RidesRepository

DRIVER

Controller

· GET pool · trips · requests · routes

· POST location · route · online · accept

· POST arrive · depart · pickup · dropoff

4 Service

· DriverService: routes, accept, trips

· TripService: stops, fare + dues at drop-off

RidesRepository

5 RidesRepository.withVehicleLock() · one vehicle = one line · used by PoolingService, RidesService, DriverService, TripService

7 BEGIN · lock_timeout 3s · SELECT vehicle FOR UPDATE · re-check R1–R4 · compare-and-set · seats ± n · refill freed seats (SKIP LOCKED) · COMMIT

4 DATA

PostgreSQL (Neon in prod · 17 in Docker locally) · Prisma 7 migrations

Identity

users (role, email/phone UQ)

driver_profiles (NID|passport)

sessions (token_hash UQ)

Geography

zones (14, code UQ)

zone_distances (km, both ways)

routes (6) · route_stops (km)

10 Fleet

vehicles (3 seats, plate UQ)

driver_id UQ · is_online

route_id · current_zone_id

Rides and pools

ride_requests (status, fares, fees)

pools (route, stop, seats, earnings)

pool_members (pickup/dropoff stop)

ride_events (history)

Guarantees in the database

· CHECK seats_taken ≤ capacity

· CHECK pickup_stop < dropoff_stop

· CHECK collected = driver + fee

· CHECK fee only on a cancelled ride

· 1 active pool per vehicle

· 1 active membership per ride

· 1 active ride per passenger

· 1 account per email · phone · ID

· 1 vehicle per plate

Money in integer paisa · times in UTC (shown in Dhaka time) · CHECK constraints and partial unique indexes are hand-written migration SQL

DELIVERY

- GitHub

feature/* → PR → merge commit
- CI · GitHub Actions

api (lint, types, unit, e2e on Postgres) · web · docker
- Render

Docker: migrate deploy → seed → start
- Vercel

Next.js build with API_URL
- Local

docker compose up: db · api · web (+ db-test)

DESIGN PRINCIPLES

- Integrity first

locks + constraints, tested with races
- Thin layers

HTTP only in controllers; rules in services
- One source of truth

the API decides; the web only shows
- Simple on purpose

no queues, Redis or microservices
- Explainable

every ride status change has an event row